

Student Marks Organizer

Submitted By

Student Name	Student ID
Rifat Ahmad Bin Sarwar	251-15-521
Maruf Al Jubaer	251-15-232
Md Khalid Hossain	251-15-267
Madiha Rahman Maha	251-15-657
Maliha Nafzan Taisha	251-15-432

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE-214:**
Algorithm Lab in the Computer Science and Engineering Department



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 23, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Shahrin Khan**, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Shahrin Khan

Lecturer

Department of Computer Science and Engineering
Daffodil International University

Submitted by

Rifat Ahmad Bin Sarwar ID:251-15-521 Dept. of CSE, DIU	
Maruf Al Jubaer ID: 251-15-232 Dept. of CSE, DIU	Md. Khalid Hossain ID: 251-15-267 Dept. of CSE, DIU
Madiha Rahman Maha ID: 251-15-657 Dept. of CSE, DIU	Maliha Nafzan Taisha ID: 251-15-432 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Analyze and calculate time complexity and space complexity of various algorithms or any written code using mathematical formula and comparison of algorithms.
CO2	Solve real-life problems by applying algorithmic paradigms like Divide and Conquer and Greedy methods.
CO3	Design and implement efficient algorithms to handle persistent data and optimize system performance.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C4	KP3	EP1
CO2	PO2	C3	KP5	EP2
CO3	PO9	C5	KP6	EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1-2
1.1 Introduction.....	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Feasibility Study.....	2
1.5 Gap Analysis.....	2
1.6 Project Outcome	2
2 Proposed Methodology/Architecture	3-4
2.1 Requirement Analysis & Design Specification	3
2.1.1 Overview.....	3
2.1.2 Proposed Methodology/ System Design.....	3
2.1.3 UI Design.....	4
2.2 Overall Project Plan	4
3 Implementation and Results	5-6
3.1 Implementation	5
3.2 Performance Analysis	5
3.3 Results and Discussion.....	6
4 Engineering Standards and Mapping	7-11
4.1 Impact on Society, Environment and Sustainability	7
4.1.1 Impact on Life	7
4.1.2 Impact on Society & Environment	7
4.1.3 Ethical Aspects.....	7
4.1.4 Sustainability Plan.....	7
4.2 Complex Engineering Problem	8
4.2.1 Mapping of Program Outcome	8
4.2.2 Complex Problem Solving	8
4.2.3 Engineering Activities.....	10

5 Conclusion	12-13
5.1 Summary.....	12
5.2 Limitation.....	12
5.3 Future Work	13
References	14

Chapter 1

Introduction

This chapter provides an overview of the Student Marks Organizer project, outlining its purpose, goals, and the gap it fills in existing academic management approaches.

1.1 Introduction

The Student Marks Organizer is a console-based academic management system built in the C programming language. It is designed to automate the collection, processing, ranking, and analysis of student marks across multiple courses. The system allows users to enter student details and course marks, after which it automatically calculates totals, averages, GPAs, and letter grades according to a standard grading rubric.

One of the standout features of this system is its use of the Merge Sort algorithm to rank students by overall GPA in descending order. This ensures the result sheet always reflects the academic standing of each student accurately. Beyond just sorting, the system also supports searching for individual students, updating marks, displaying class statistics, and persisting all data through file handling. Together, these features make the system a practical, algorithm-driven solution for academic result management.

1.2 Motivation

Academic institutions often face challenges when managing student results manually. Spreadsheet tools like Microsoft Excel require considerable effort to maintain and are prone to formula errors. Result processing that involves sorting students by GPA or computing class averages becomes especially tedious when done by hand, particularly as the number of students grows.

The motivation behind this project was to build a lightweight, automated system that applies core algorithm concepts — particularly sorting and searching — to solve a practical academic problem. The team was inspired to demonstrate that foundational algorithms, when properly implemented, can dramatically reduce the effort and error involved in managing student records.

1.3 Objectives

The primary objectives of the Student Marks Organizer are as follows:

- To design and implement a student record management system driven by algorithmic logic.
- To apply the Merge Sort algorithm for ranking students by GPA in descending order.
- To compute total marks, averages, GPA, letter grades, and pass/fail results automatically.
- To provide search and update functionalities to manage individual student records efficiently.
- To display class-level statistics such as the highest, lowest, and average GPA.
- To implement file handling for persistent data storage across program sessions.

1.4 Feasibility Study

A feasibility study evaluates whether the Student Marks Organizer can be realistically developed, implemented, and maintained within the available resources. It considers technical, economic, operational, and scheduling aspects to determine the practicality of the project.

1.5 Gap Analysis

In many academic institutions, student result processing is still done manually or through basic spreadsheet tools. While spreadsheets offer some automation, they fall short when it comes to automatic sorting by performance, dynamic GPA computation based on a grading scale, and persistent storage in a unified record system.

The Student Marks Organizer bridges this gap by integrating an automated grading engine with a sorting mechanism. Instead of requiring a teacher to manually rank students or calculate GPAs, the system handles all of this internally. The use of Merge Sort ensures that the ranking is always accurate and efficient, even as the number of students grows. The system also supports searching and updating records — operations that are tedious in a manual setup but instant in this system.

In short, the gap between slow, error-prone manual result management and the need for an automated, algorithm-based solution is precisely what this project addresses. It delivers a structured, reliable, and easy-to-use system that outperforms traditional approaches in speed, accuracy, and consistency.

1.6 Project Outcome

The Student Marks Organizer was successfully implemented and delivers all the planned functionalities. The system is capable of handling up to 100 students across a configurable number of courses. It automatically computes each student's total marks, average, GPA, and letter grade, and applies a fail condition for any student scoring below 40 in any individual course.

Students are ranked by GPA using Merge Sort, and the ranked result sheet is displayed in a clean, formatted table. The search function allows users to instantly retrieve a student's academic profile by ID. The update function re-triggers the sort after any modification, keeping the ranking always current. The statistics module provides class-level insights at a glance. Finally, all data can be saved to a file and reloaded across sessions, making the system usable beyond a single run.

Overall, this project demonstrates how algorithm knowledge acquired in the classroom can be applied to build a functional, real-world tool. It also strengthens the team's practical skills in C programming, algorithmic design, and software development.

Chapter 2

Proposed Methodology/Architecture

This chapter discusses the overall structure, workflow, and design approach of the Student Marks Organizer. It outlines how the system operates, how different algorithmic modules interact, how data is managed, and how different modules interact with each other to ensure a smooth user experience.

2.1 Requirement Analysis & Design Specification

The analysis helps identify what the system must achieve, while design specification outlines how those requirements will be implemented. This section explains the functional and non-functional needs of the Student Marks Organizer along with the architecture that supports its operations

2.1.1 Overview

The Student Marks Organizer is designed to eliminate manual result processing by providing an automated, algorithm-driven system. The user enters student names, course names, and marks, and the system computes all academic metrics automatically. The core data structure is a fixed-size array of student records, each holding the student ID, name, course marks, total, average, GPA, grade, and result. All ranking is done using Merge Sort to ensure efficiency and correctness.

2.1.2 Proposed Methodology/ System Design

The proposed system follows a modular and structured design approach. When the user selects Option 1 (Input Students), the system first collects the number of students and courses, then prompts for course names and each student's marks. After all input is received, the `calculate()` function computes totals and averages, and assigns grades. The `mergeSort()` function is then called to rank all students by GPA in descending order. For subsequent operations — Display, Search, Update, Statistics, Top Performers — the system reads from the sorted student array. The Save and Load options interact with the file system to persist and restore data between sessions.

The algorithm at the core of this workflow is Merge Sort. It works by recursively dividing the student array into two halves, sorting each half independently, and then merging them back in descending order of GPA. This divide-and-conquer strategy guarantees a time complexity of $O(n \log n)$, which is significantly more efficient than simpler quadratic sorting methods for larger datasets.

Overall, the system design focuses on simplicity and ease of use. By dividing operations into logical modules—authentication, mark management, data storage, and user interaction—the system provides a clear workflow that ensures reliable and organized management of student information.

2.1.3 UI Design

```
===== Student Marks Organizer =====  
1. Input Students  
2. Display Result Sheet  
3. Search Student  
4. Update Student  
5. Top Performers  
6. Class Statistics  
7. Save Data  
8. Load Data  
9. Exit  
Choose Option: |
```

The user interface is a simple text-based menu shown in the console. The main menu provides 9 options. Each option triggers a specific operation. For example, selecting Option 1 initiates a guided input process where the user is prompted to enter the number of students, number of courses, course names, student names, and marks one by one.

The result display (Option 2) presents a formatted table with student IDs, names, course marks, average, GPA, grade, and result, all aligned in columns for readability. Options for search and update prompt the user to enter a student ID, after which the system either retrieves or modifies the corresponding record. Input validation is applied where applicable — for example, the update function recalculates and re-sorts after any change, ensuring the result sheet remains consistent.

2.2 Overall Project Plan

The overall project plan for the Student Mark Organizer was executed in four main phases: requirement analysis, design, implementation, and testing. In the analysis phase, the team identified the key features needed — student input, marks calculation, sorting, searching, updating, statistics, and file handling. During the design phase, the function structure and data flow were planned before writing any code.

The implementation phase involved writing the C code modularly — each feature as its own function — and integrating them through the main menu switch-case structure. The Merge Sort algorithm was implemented and tested separately before integration. The testing phase involved running multiple test cases, including edge cases such as a student failing a single course while performing well overall, or loading data from a previously saved file. All test scenarios produced the expected results.

The overall timeline fit within a single semester, and the task division among the five team members was organized around the modules, with each person responsible for specific functions and documentation sections.

Chapter 3

Implementation and Results

This chapter describes how the Student Management System was developed and evaluates how effectively the system performs after implementation. It highlights the programming techniques, the algorithm implementation followed by an analysis of the system's performance based on accuracy, speed, and usability.

3.1 Implementation

The system is implemented in standard C using a single source file. All student data is stored in an array of Student structures, with each structure holding the student ID, name, marks for all courses, total, average, GPA, grade, and result. The key algorithmic component is the Merge Sort function, which operates directly on this array.

The grading logic is handled by the assignGrade() function. It first checks whether any individual course mark falls below 40 — if so, the student immediately receives a failing grade regardless of their average. Otherwise, the average is used to determine the GPA and letter grade on a standard scale ranging from A+ (GPA 4.00, average ≥ 80) down to D (GPA 2.00, average ≥ 40). This mirrors the grading policy commonly used at Bangladeshi universities.

The Merge Sort algorithm (mergeSort() and merge()) divides the student array recursively and merges in descending GPA order. The use of a temporary array during the merge phase ensures correctness while keeping the implementation clean. After every input or update operation, the array is re-sorted to maintain the ranking at all times.

File handling is implemented through the save() and load() functions. Student records are written to a plain text file (students.txt) in a structured format that can be read back and fully reconstructed — including recalculating all derived values and re-sorting — on the next load. ID generation uses a global counter (seq) that is also saved and restored to prevent duplicate IDs across sessions.

3.2 Performance Analysis

The Student Marks Organizer was evaluated on three dimensions: algorithmic efficiency, accuracy, and usability.

From an algorithmic standpoint, the use of Merge Sort provides $O(n \log n)$ time complexity for sorting, which is optimal for comparison-based sorting. This is a significant improvement over bubble sort or selection sort (both $O(n^2)$), especially as the number of students grows. Searching is performed through a linear scan of the array, giving $O(n)$ complexity — acceptable for the scale of this system. Grade computation runs in $O(n \times c)$ time, where n is the number of students and c is the number of courses, which is inherently necessary.

Accuracy was verified through a series of test cases. These included: a student failing one course while holding a high average in others (correctly assigned F grade), a student with a borderline average of exactly 80 (correctly assigned A+), and records saved and reloaded across two program sessions (correctly reconstructed with the same ranking). In all cases, the system produced the expected output.

Usability is straightforward: the menu-driven interface guides users through each step with clear prompts. The formatted result sheet is easy to read, and options like search and statistics are accessible with minimal input. Users with basic C program experience can operate the system without any learning curve.

3.3 Results and Discussion

The Student Marks Organizer was executed successfully in all test scenarios. The program's main menu clearly presents all available options, and each operation performed correctly during testing:

- Entering students and courses produced a ranked result sheet sorted by GPA in descending order.
- The result sheet accurately showed totals, averages, GPAs, grades, and pass/fail status for each student.
- The search function retrieved the correct student profile when a valid ID was entered, and printed 'Student Not Found' for invalid IDs.
- The update function modified marks, recalculated all metrics, and re-sorted the list correctly.
- The top performers list correctly displayed the first N students from the sorted array.
- The statistics function correctly identified the highest average, lowest average, and overall class average.
- Saving and loading data worked correctly, with the ranking preserved after reload.

The results confirm that the system meets all its stated objectives. The integration of Merge Sort as the backbone of the ranking system ensures that the result sheet is always accurate and ordered, which is the central requirement of the project. The modular code structure also makes the system easy to extend — additional features such as CGPA tracking across semesters or export to CSV could be added without restructuring the existing codebase.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the broader impact of the Student Marks Organizer on society, the environment, ethical considerations, and sustainability. It also presents project management aspects, cost analysis, and how the system aligns with engineering program outcomes. Additionally, it explores complex engineering problems addressed by the system and provides rationales for problem-solving approaches.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The Student Marks Organizer simplifies a time-consuming academic task — compiling and ranking student results — by automating it through algorithmic computation. Faculty and administrators benefit from reduced manual effort, faster result processing, and fewer errors. Students benefit from transparent, consistent grading that is applied the same way every time. The system also makes it easy to identify top performers and at-risk students quickly, which supports better academic decision-making.

4.1.2 Ethical Aspects

Digitizing student result management reduces the need for paper-based records, printed result sheets, and physical registers, which contributes to environmentally responsible academic administration. At a broader level, providing students with accurate and timely academic feedback supports educational quality and institutional trust. Systems like this one, when adopted at scale, can contribute to a culture of transparency and accountability in higher education.

4.1.3 Sustainability Plan

The system applies a uniform grading algorithm to every student, eliminating subjective bias in grade assignment. The auto-generated student IDs and structured result display ensure consistency and fairness. Student data is stored in a plain text file with no unauthorized access mechanisms built in at this stage — however, the architecture is straightforward enough that role-based access or encryption could be added in a future version. The system handles academic data with integrity, ensuring that no marks can be silently altered without going through the update function, which recalculates and re-logs all changes.

4.2 Complex Engineering Problem

The development of a secure, efficient, and user-friendly student marks organizer system addresses multiple complex engineering problems, including secure authentication, persistent data storage, modular programming, and error handling in a real-time environment.

4.2.1 Mapping of Program Outcome

The system aligns with targeted Program Outcomes (POs) as follows:

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	Applied in-depth engineering and algorithmic knowledge by implementing Merge Sort — a divide-and-conquer algorithm — to rank students by GPA. The team analyzed time complexity ($O(n \log n)$) and selected this algorithm over simpler alternatives based on efficiency considerations.
PO2	Identified the limitations of manual and spreadsheet-based result management and formulated an algorithmic solution. The system's grading engine, sort mechanism, and search function each represent a well-reasoned algorithmic design decision based on problem requirements.
PO9	The project was completed collaboratively by a five-member team with diverse academic backgrounds. Each member contributed to a different module of the system, demonstrating effective teamwork, communication, and the ability to integrate independently developed components into a cohesive product.

4.2.2 Complex Problem Solving

The purpose of this section is to identify the complex engineering problems in the Student Marks Organizer and map them to appropriate problem-solving categories, while also providing the knowledge profile and rationale for each problem.

Table 4.2: Mapping with complex problem solving.

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stakeholder Involvement	EP7 Inter- dependence
✓	✓	✓				

EP1 – Depth of Knowledge Required

Justification:

The project moves beyond basic programming by requiring an in-depth understanding of the Divide and Conquer paradigm. Implementing Merge Sort requires knowledge of recursive function calls and the management of the system stack. Unlike simple algorithms like Bubble Sort, Merge Sort necessitates a mathematical understanding of time complexity to ensure the system remains performant as the student database grows. Furthermore, managing an array of Structures rather than simple integers requires a deep grasp of memory mapping and pointer manipulation within the merge function to avoid data corruption.

EP2 – Range of Conflicting Requirements

Justification:

A core engineering conflict exists between Data Persistence and Execution Speed. Storing data in a flat text file (students.txt) is slow due to Disk I/O, but necessary for data safety. To resolve this, the system is designed to load data into the RAM (Array of Structures) only once, perform all complex sorting and calculations in-memory for maximum speed, and then sync back to the disk. Additionally, the system must balance Sort Stability (keeping records in a predictable order) with Memory Constraints, as Merge Sort requires extra space to store the temporary temp[100] array during the merging phase.

EP3 – Depth of Analysis

Justification:

The project requires a rigorous analysis of edge cases, such as handling a "Fail" condition in GPA calculation. If a student fails a single course, the entire GPA must be overridden to 0.00 regardless of the average. This necessitates a multi-layered logic check within the assignGrade function before the sorting algorithm is even triggered. Analyzing how the sorting algorithm behaves when multiple students have identical GPAs (Stability) is critical to ensuring the "Ranked Result Sheet" is fair and technically sound.

4.2.3 Engineering Activities

The purpose of this section is to map the engineering activities involved in the Student Marks Organizer and show how each activity contributes to solving specific problems. It also provides the rationale for each activity to justify its relevance and impact.

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	✓	

EA1 – Range of Resources

Justification:

The activity involves the synthesis of multiple computational resources. It utilizes the **C Standard Library** for core logic, **File System API** for database persistence, and modern **Integrated Development Environments (IDEs)** like Code::Blocks or VS Code for debugging the recursive logic. The use of modular programming—separating the code into input, calculate, mergeSort, and save—demonstrates an engineering approach to resource organization and code maintainability, which is essential for professional software development.

EA2 – Level of Interaction

Justification:

This project demonstrates a high level of interaction between different mathematical and technical modules. The calculate module must feed accurate data into the mergeSort module, which in turn must interact with the display and save modules. A failure in the logic of the merge function directly impacts the integrity of the "Top Performers" output. This interdependence requires the developer to act as a system architect, ensuring that data flows seamlessly and accurately between the user interface (Menu) and the back-end algorithmic engine.

EA3 – Innovation

Justification:

While sorting is a standard task, implementing a Ranked Statistics Module (Highest, Lowest, and Average) provides an innovative layer of data analytics for academic administrators. The activity also addresses professional responsibility by ensuring Data Integrity. The generateID function automates ID creation to prevent duplicate records, showcasing an engineering solution to a common administrative error. By providing a "Load" and "Save" mechanism, the activity mimics real-world enterprise software where data loss is not an option.

EA4 – Consequences for Society and Environment

Justification:

The Student Marks Organizer fulfills EA4 by contributing positively to society and the environment. Digitizing student marks management reduces dependency on paper-based records, thereby supporting environmentally sustainable practices. The system improves accuracy, transparency, and efficiency in academic result processing, which benefits students, teachers, and academic institutions. By minimizing manual errors and providing quick access to academic performance data, the system enhances trust and reliability in educational record keeping. These societal and environmental benefits demonstrate compliance with EA4 – Consequences for Society and Environment.

Chapter 5

Conclusion

This chapter presents a summary of the Student Marks Organizer project, highlights its limitations, and suggests areas for future improvements. It reflects on the overall achievements and learning outcomes from the project.

5.1 Summary

The Student Marks Organizer successfully demonstrates the practical application of algorithm design principles — specifically the Merge Sort algorithm — to the real-world problem of academic result management. The system automates the entire result processing pipeline: from data entry and marks computation to grade assignment, GPA ranking, and statistical analysis. It handles up to 100 students across multiple configurable courses, applies an institutional grading rubric, and maintains a persistent record through file handling.

The project was completed in C using modular programming, with each feature encapsulated in its own function. The use of Merge Sort for ranking ensures $O(n \log n)$ efficiency, while the linear search and other supporting functions operate correctly within the expected scale. Overall, the system meets all its stated objectives and serves as a practical demonstration of how algorithms can be applied to build useful, real-world software.

5.2 Limitation

Despite its effectiveness, the Student Marks Organizer has several limitations that are worth acknowledging:

- The system is console-based and does not include a graphical user interface (GUI), which limits accessibility for non-technical users.
- Data is stored in a plain text file without encryption or access control, making it unsuitable for sensitive institutional data without further security measures.
- The system can only handle one class at a time per session; managing multiple sections or semesters simultaneously is not supported.
- The student array has a fixed upper limit of 100 students, which may be insufficient for large classes.
- There is no support for weighted marks (e.g., assignments vs. exams) or credit-based GPA computation.
- The search function uses a linear scan, which becomes slower as the number of students grows significantly.

5.3 Future Work

Several improvements and extensions are planned or recommended for future versions of the system:

- Implementing a Binary Search Tree (BST) or hash table for faster student search operations.
- Replacing the flat text file storage with an embedded database such as SQLite for better scalability and data integrity.
- Developing a graphical or web-based frontend to improve usability for non-technical administrators.
- Adding support for multiple sections, semesters, and course credit weightings in GPA computation.
- Implementing user authentication to control access and protect sensitive student records.
- Adding export features to generate PDF or CSV result sheets for official use.
- Extending the statistics module to include grade distribution charts and trend analysis across semesters.

References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
2. Jon Kleinberg and Eva Tardos, Algorithm Design, Pearson Education India, 2006.
3. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

